

Monopoly Deal Cards Game - Phase 1 Documentation

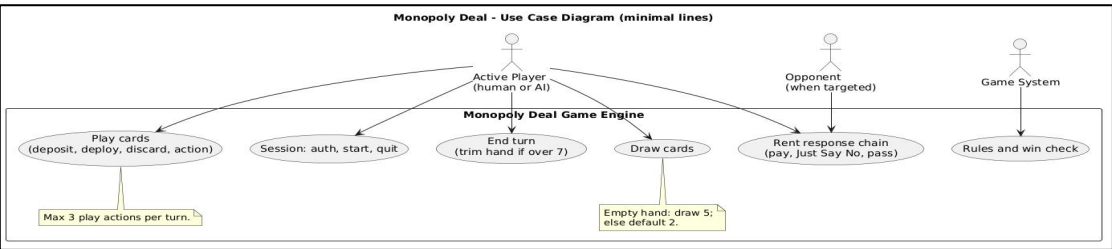
Group 7

1. UML Design of the project

Monopoly Deal is a fast-paced strategy card game based on the classic Monopoly theme, designed for 2 to 5 players. Players aim to draw cards from the shared deck, manage their hand of cards, and use action cards strategically to be the first to collect three complete, distinct color-coded property sets during their turn and win the game. The core game mechanics include property building, rent collection, bank deposits, and strategic disruption of opponents' assets. It is designed to provide players with a smooth and competitive digital gaming experience via a graphical user interface (GUI).

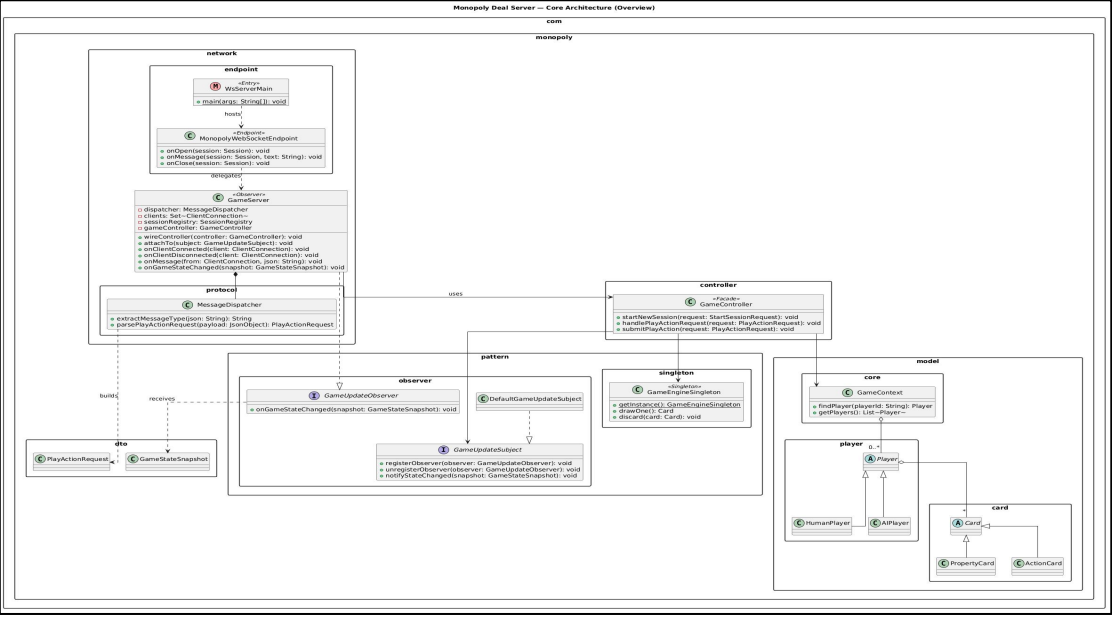
1.1 Use Case Diagram

This diagram defines the functional scope of the system and illustrates how the three types of participants—the current player (human or AI), opponents, and the game system—interact by performing core use cases including drawing cards, playing action cards, and win/loss judgment.



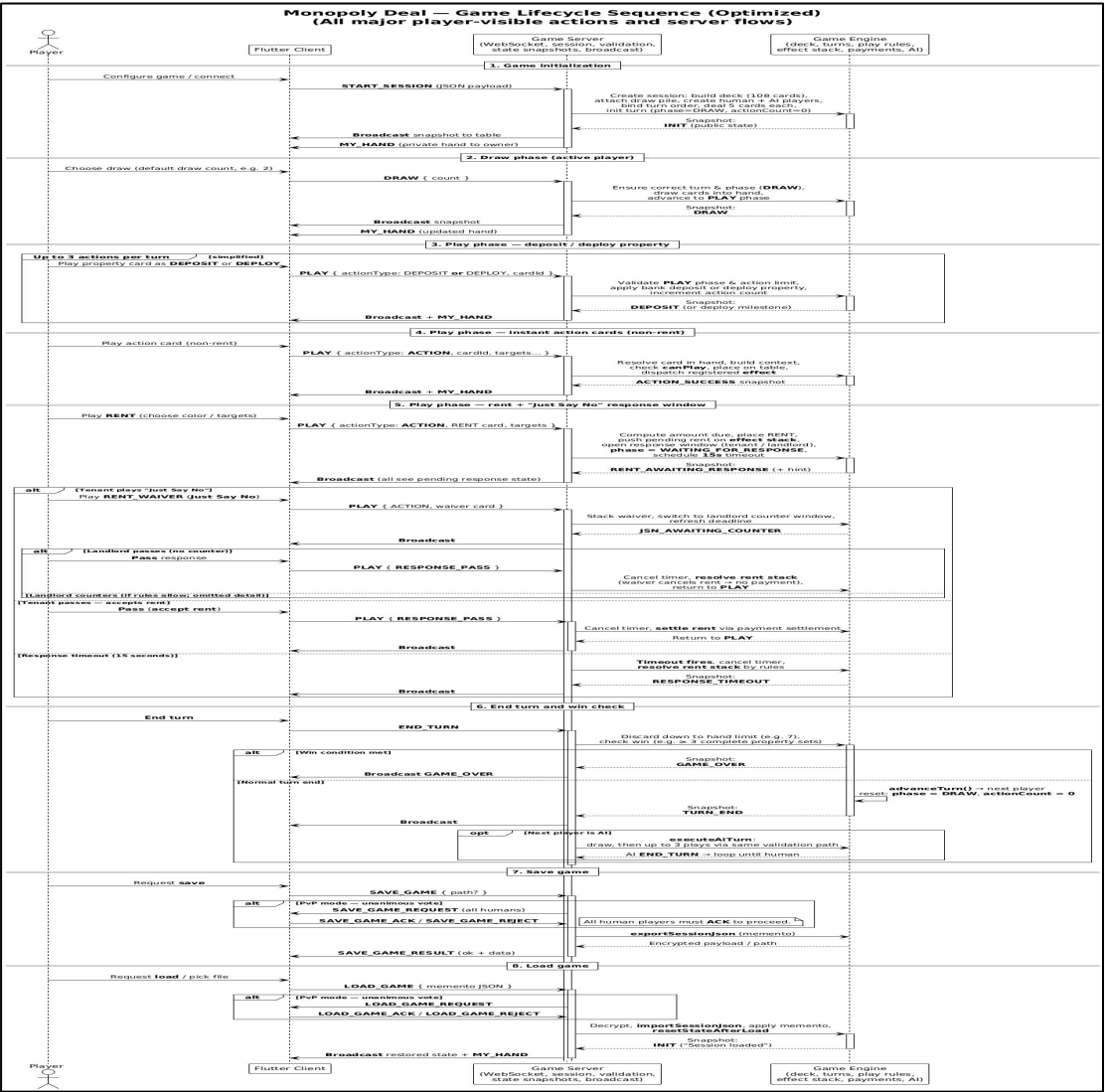
1.2 Class Diagram

The class diagram presents the static structure and architecture design of the system. By adopting the MVC (Model-View-Controller) pattern, it clearly describes the hierarchical relationships and interaction contracts among the property cards, action cards, player logic, and network communication modules.



1.3 Sequence Diagram

The sequence diagram captures the dynamic behavior throughout the game life cycle, and meticulously documents the message flows exchanged in chronological order between the client, server, and game engine — from game initialization and turn logic switching to the rent processing mechanism.



2. User Requirement

- Game Mode: The system supports 2-5 players, allowing free selection between AI opponents and real players.
- Interactive experience: Players should be able to perform card drawing, playing, rent payment, and property management through the graphical user interface (GUI).
- Information transparency: Players should be able to view real-time public deck status, their own assets, and their opponents' disclosed assets, while maintaining privacy of their hand cards.
- Automatic judgment: The system shall automatically perform complex rent collection calculations and rule verification, and immediately notify the player when victory conditions are met in player's turn.
- Progress Management: The system shall allow players to save game progress across different modes for later restoration.

3. System Requirement

3.1 Functional Requirement

3.1.1 Game Basic Startup and Initialization

- a. Mode selection: The system shall provide entry points for both "human vs.machine"and "player vs. player" modes. In human vs. machine mode, players shall be able to select three AI difficulty levels: Easy, Normal, and Hard.
- b. Number of players and turn order: The system supports 2-5 players to create a new game, randomly assigning one player as the first mover with clockwise turn rotation.
- c. Resource initialization: When the game starts, the system shall:
 - Generate a deck using the standard 108-card configuration.
 - Automatically deal 5 initial cards to each player.
 - Display the remaining cards in the deck and initialize an empty discard pile.
- d. Rule view: The system shall integrate a game rules panel, allowing players to access and view them anytime in-game.

3.1.2 Turn Logic and Action Control

- a. Card Drawing Mechanism: During their turn, players must first draw 2 cards from the shared deck. If the player's hand is empty before drawing, the system must allow them to draw 5 cards. After drawing, the system automatically updates the player's hand count and refreshes the hand display area in real time.
- b. Card Drawing Function: After drawing cards, players can choose to deposit them into their "Property Zone" or "Action Zone", or deposit them into their personal bank account. A maximum of three cards can be drawn per turn (cumulative across all drawing actions). Supported operations include:
 - Bank Deposit: Players can deposit money cards and action cards into their personal bank stacks. After depositing, the action cards lose their original functions and are only used for currency settlement.
 - Property Placement: Players can place property cards from their hand into their property zone to build a matching property set. The system automatically displays property zone cards by color.
 - Action Execution: Allows players to draw action cards from their hand to the central action zone, triggering corresponding special effects (such as rent collection, card theft, rent waiver, etc.). The system automatically executes the effects and updates the game status after drawing.
- c. Round End and Discard: After completing all actions including drawing and playing cards, the player enters the discard phase. If the player's hand exceeds 7 cards, the system will automatically force them to discard cards into the bottom of the deck until the hand reaches 7. Upon completion of discarding, the system terminates the current player's turn and proceeds to the next player.

3.1.3 Core Game Mechanisms

- a. Rent collection logic:
 - The system automatically calculates rental fees based on the completion status of matching property sets (basic properties, houses with attached properties, or hotels with attached properties) in players' property zones, in accordance with game rules, and sends rental collection requests to designated target players.
 - Payment processing:

- ◆ The system allows players to settle rent arrears by returning money cards from their personal bank or property cards from the assets section to the Rent Collector's hand, but direct payment using hand cards is prohibited.
- ◆ The system must enforce the 'no-change return' policy: if a player's card payment exceeds the total rent due, the excess amount is not refunded but directly retained by the rent collector.
- ◆ If a tenant's personal bank has no available money cards and their property area contains no refundable property cards, they are exempt from paying the current rent, and the tenant's rent collection attempt will fail.

b. Property Wild Card Mechanics

The Property Wild Card can substitute for any color property card to complete matching property sets. Players may freely adjust the position of the Property Wild Card in their property zone during their turn. Upon adjustment, the system immediately recalculates the completion status of the corresponding property set and updates the rental calculation criteria.

c. Card legality verification:

- The system performs real-time rule validation on all player actions, immediately blocking illegal moves such as playing rent collection cards without matching property sets, exceeding the three-card per turn limit, or discarding hands when empty. Clear error messages are displayed, for example: 'No available property sets currently available to play rent collection cards.'
- When players select action card targets, the system verifies their legitimacy (e.g., stealing cards requires selecting players with hand cards or bank cards, demolishing properties requires selecting players with deployed properties) and blocks invalid targets.

d. Drawing deck replenishment mechanism:

When players draw cards, if the public deck has no available cards, the system automatically reshuffles all cards from the discard pile and converts them into a new public deck to ensure smooth gameplay. After replenishing the deck, the remaining quantity of the new public deck is displayed to all players.

3.1.4 Game Status Feedback and Operation Tips

a. Real-time status feedback:

- The system displays in real-time on the game interface the current round players, remaining cards in the deck, and discard pile count.
- Each player's interface displays real-time information including their hand size, property area cards (categorized by color), personal bank stack count, and total face value.
- Public information of other players (property zone cards and total bank stack value) is displayed normally. Hand cards are private information, showing only quantities without displaying specific cards.

b. Instant prompt for operations:

- After players perform actions like drawing cards, playing cards, or discarding cards, the system provides textual prompts and card animations to display the results (e.g., "Player A draws 2 cards, current hand count: 6" or "Player B initiates rent collection from Player C, rent amount: 5").
- When players have executable actions (such as drawing cards or playing cards), the system highlights the corresponding action buttons to guide them.

3.1.5 Game Victory and End Judgment

a. Victory determination:

The system continuously monitors all players' property zone statuses. If a player collects three or more complete property sets during their own turn, they will be immediately declared the winner; if the victory condition is met during an opponent's turn, the system will only declare victory and end the game when it becomes the player's next turn.

b. Force termination judgment:

If all players voluntarily quit the game or a single game session exceeds 1 hour, the system will forcibly terminate the game and display the current progress (including each player's property collection status and total bank balance).

3.1.6 Game Progress Management

a. Human vs. Machine Mode

- Pause: Players can pause the game at any time to freeze the interface. After pausing, player can continue, save and exit, or abandon and exit without additional confirmation steps.

- Save progress: Supports manual save (available anytime) + automatic save (automatically saves progress after every 3 full rounds); When starting the game, player can directly load locally saved progress and restore all game states (player/AI/deck/round) with one click. The system shall provide a 'Load Game' entry in the main menu to reconstruct the game state (including deck, hands, and assets) from local encrypted files while ensuring version compatibility.

b. Player vs. Player (Multiplayer)

- Pause Request and Timeout Mechanism:
 - ◆ The system shall allow only the player whose turn is active to initiate a pause request, which requires unanimous confirmation from all players to take effect.
 - ◆ To maintain consistency and prevent system deadlocks, the system shall implement a 30-second confirmation timer.
 - ◆ If any player fails to respond within this window, the system shall automatically reject the pause request and resume the current turn.
- Abnormal Disconnection Handling (Robustness):
 - ◆ The system shall monitor the connection status of all players and, upon detecting a disconnection, initiate a 60-second reconnection grace period.
 - ◆ If the player fails to reconnect within the time limit, the system shall prompt the remaining players to vote for either "AI Takeover" or "Forced Save and Terminate" to ensure the session does not remain in an unrecoverable state. The difficulty of AI will be normal.
- Exit and Progress Integrity:
 - ◆ Following a successful pause, the system shall restrict options to "Continue Game for All" or "Save and Exit for All" to prevent individual abandonment.
 - ◆ However, if a session is forcibly terminated due to a "Force Termination Judgment" (e.g., exceeding 1 hour), the system shall ensure that data integrity is maintained by automatically archiving the current property collection status and bank balances for all participants.

3.2 Non-Functional Requirements

3.2.1 Product Requirements

a. Usability:

- Audience adaptation: The GUI design shall align with the cognitive level of children aged 8 and above, featuring simple and intuitive operation logic.

- **Interactive feedback:** The system must provide visual or textual feedback within 0.5 seconds after a user performs any legal or illegal action (e.g., playing the fourth card).
- **Visual adaptation:** Color-coded property cards include text labels for color names to prevent color-blind users from encountering recognition difficulties.
- **Newbie Guide:** The game automatically launches a distributed tutorial upon first startup. Players can complete the core process of card drawing, playing, and rent collection through pop-up windows and highlighted guidance, with the option to skip steps at any time.

b. **Performance:**

- **Response speed:** The combined decision-making time for AI players and UI animation rendering time must not exceed 1.5 seconds to ensure a 'fast-paced trading' gaming experience.
- **Concurrent processing:** In multiplayer battle mode, the system must reliably support up to 5 players interacting online simultaneously without noticeable latency.
- **Processing efficiency:** The single operation time for drawing, shuffling, and automatic rent calculation must not exceed 0.3 seconds, ensuring smooth interface performance without lag.
- **Network communication latency:** In multiplayer online battle mode, WebSocket JSON message transmission and state synchronization latency between the client (Java FX) and server (Java) must be kept under 200 ms to ensure state consistency across multi-platform interfaces.

c. **Robustness and Reliability**

- **Error handling:** The system must identify and block all non-compliant inputs (e.g., attempts to pay with insufficient balance) without causing program crashes.
- **Data integrity:** During gameplay, the system must ensure that automatically saved progress files (Auto-save) remain intact in case of network fluctuations or unexpected system shutdowns.
- **Card data consistency:** The game maintains a constant total of 108 standard card piles throughout. Real-time pile count verification occurs during drawing, discarding, and replenishment to prevent card duplication or loss.

3.2.2 Organizational Requirements

a. Development Requirements:

- Programming languages and technology stack: The system adopts a front-end and back-end separation architecture. The core backend logic and servers must be developed using Java (JDK 17 or later versions). The front-end graphical user interface (GUI) is built with the JavaFX framework, utilizing FXML for declarative UI layout. This stack ensures a strict adherence to the MVC (Model-View-Controller) pattern, delivering a high-performance desktop experience with seamless animated interactions.
- Architecture pattern: The MVC (Model-View-Controller) pattern must be adopted to achieve separation of logic and interface.
- Design pattern implementation: The system must incorporate five design patterns appropriately (including Singleton, Facade, Observer, etc.).
- Version control: The team must collaborate using Git, with commit messages clearly documenting changes. Contributions from all five team members should be proportionally balanced.
- Design quality: The code structure must adhere to the "clean design" principle, prohibiting the use of "big classes" or "long methods," and ensuring that each class implements only a set of related functionalities.
- Maintainability: Proper interfaces must be defined to handle classes with similar behaviors but no inheritance relationships.

b. Delivery constraints (Environmental Requirements):

- Running environment: The program must run on standard operating systems (Windows/macOS/Linux) with JDK 17 or later installed.
- Cross-platform compatibility: Works seamlessly on Windows, macOS, and Linux systems without interface garbled text or font display issues, while maintaining consistent card images and button styles

3.2.3 External Requirements

Software Engineering Guidelines (Legislative/Ethical Requirements):

- Confidentiality: The system must ensure that in multiplayer mode, Player A cannot access Player B's hand information through any unauthorized means.
- Local data security: Player progress files (including hand cards, assets, and banking information) are stored using lightweight encryption to prevent tampering with game data through manual file access. In multiplayer mode, players' public information is synchronized solely via the game interface, eliminating risks of local data leakage.

4. Project Completion Plan

Project Completion Plan (Weeks 1-15)

Phase-1 due Week 8 (2026-04-22). Phase-2 due Week 15 (2026-06-09). Demo on 2026-06-11/12.

Week	Key Milestones (focus shifts from analysis/UML in Weeks 1-8 to implementation/testing in Weeks 9-15)
1	Understand the game, project kickoff, requirement elicitation.
2	Design the schedule plan and complete the team division of labor, Elicit functional/non-functional requirements.
3	Elicit functional/non-functional requirements.
4	Discussion, understanding, learning, designing system architecture (MVC).
5	Design system architecture, Draft UML diagrams (Class, Sequence, Use Case).
6	Draft UML diagrams (Class, Sequence, Use Case).
7	Implement basic Model classes (Card, Player).
8	Finalize UML & Phase-1 PDF Report, Submit Phase-1 to Brightspace.
9	Implement GameController and TurnManager.
10	Apply Factory (Deck creation) and Strategy (AI logic), Write JUnit test cases for game rules (e.g., Rent calculation).
11	Implement GameUpdateSubject (Observer Pattern).
12	Setup WebSocket/Network dispatchers, Connect Java Backend with JavaFX (FXML) desktop UI.
13	Refine AI difficulties (Easy/Hard), Code refactoring to eliminate "Code Smells" (Ensure no large classes/methods).
14	End-to-end system testing.
15	Write Phase-2 final PDF explanation, Code freeze and final submission.

5. Task Allocation

Name	Role	Primary Tasks	Contribution (%)
Liu Zongrun	Architectural Designer	The overall concept and structure of the design project	20%
Lei Yurun	Graphic Design	Draw UML diagrams and organize group discussions	20%
Wang Tingdong	Summary of Requirements	Summarize the requirements after the group discussion and prepare the requirements document	20%
Qiu Siqi	Review and verification	Responsible for checking the compatibility between UML diagrams and the project	20%
Liu Yuhao	Contact Assistance	Assist in summarizing the requirements document and reviewing the presentation document	20%